

 C++ >

```
```C++
```

```
/*
```

```
ĐỀ BÀI:
```

Một bộ công cụ vẽ hình không gian sử dụng hệ trục tọa độ Descartes (Oxyz) làm chuẩn có thể vẽ điểm (Point), mặt phẳng (Plane) và vectơ (Vector).

- Mỗi điểm có các thành phần thực: hoành độ x, tung độ y và cao độ z. Ví dụ: Điểm A(1;2;3) có các thuộc tính  $x = 1$ ,  $y = 2$ ,  $z = 3$ .

- Mỗi vectơ có các thành phần thực: a, b, c. Ví dụ: Vectơ  $n = (2;3;4)$  có các thuộc tính  $a = 2$ ,  $b = 3$ ,  $c = 4$ .

- Mỗi mặt phẳng có các thành phần thực A, B, C, D được biểu diễn dưới dạng phương trình tổng quát  $Ax + By + Cz + D = 0$ . Một mặt phẳng có 1 vector pháp tuyến.

Ví dụ: mặt phẳng (P):  $2x + 3y + 4z - 5 = 0$  có các thuộc tính  $A = 2$ ,  $B = 3$ ,  $C = 4$ ,  $D = -5$ .

Nếu tọa độ của một điểm thỏa mãn phương trình mặt phẳng, ta gọi điểm đó thuộc mặt phẳng đó.

Hãy cài đặt các lớp bao gồm thuộc tính và phương thức để đáp ứng chương trình chính sẽ thực hiện như sau:

1. Tạo 01 mặt phẳng.
2. Tạo 02 điểm thuộc mặt phẳng (P) và một điểm ngẫu nhiên (tự chọn).
3. Cho 02 điểm thuộc mặt phẳng đó liên kết với mặt phẳng (P) theo quan hệ thuộc mặt phẳng.
4. Hiển thị thông tin của mặt phẳng gồm có tên, thuộc tính, vectơ pháp tuyến của mặt phẳng và các điểm thuộc mặt phẳng đó.
5. Kiểm tra điểm ngẫu nhiên ở bước 2 có thuộc mặt phẳng (P) không. Nếu có thì liên kết điểm đó với mặt phẳng (P), nếu không thì tính khoảng cách từ điểm đó đến mặt phẳng.

```
*/
```

```
#include <iostream>
```

```
#include <string>
```

```
#include <list>
```

```
#include <cmath>
```

```
using namespace std;
```

```
class Plane;
```

```
class Vector {
```

```
private:
```

```
 double a, b, c;
```

```
public:
```

```
 Vector(double anA = 0, double anB = 0, double anC = 0) : a(anA), b(anB), c(anC)
```

```
{}
```

```
 void display() const {
```

```
 cout << "(" << a << "; " << b << "; " << c << ")";
```

```
 }
```

```
};
```

```

class Point {
private:
 string name;
 double x, y, z;
 list<Plane*> planes;

public:
 Point(string aName, double anX, double aY, double aZ)
 : name(aName), x(anX), y(aY), z(anZ) {}

 string getName() const { return name; }
 double getX() const { return x; }
 double getY() const { return y; }
 double getZ() const { return z; }

 void attachPlane(Plane* aPlane) { planes.push_back(aPlane); }
 void detachPlane(Plane* aPlane) { planes.remove(aPlane); }

 void display() const {
 cout << name << "(" << x << "; " << y << "; " << z << ")";
 }
};

class Plane {
private:
 string name;
 double A, B, C, D;
 Vector* normalVector;
 list<Point*> points;

public:
 Plane(string aName, double anA, double aB, double aC, double aD)
 : name(aName), A(anA), B(aB), C(aC), D(aD) {
 normalVector = new Vector(anA, aB, aC);
 }

 ~Plane() {
 delete normalVector;
 points.clear();
 }

 string getName() const { return name; }

 bool contains(Point* p) const {
 if (!p) return false;
 return (A * p->getX() + B * p->getY() + C * p->getZ() + D) == 0;
 }

 double distanceTo(Point* p) const {
 if (!p) return -1;
 return abs(A * p->getX() + B * p->getY() + C * p->getZ() + D) / sqrt(A *
A + B * B + C * C);
 }
}

```

```

bool addPoint(Point* p) {
 if (!p || !contains(p)) return false;
 points.push_back(p);
 p->attachPlane(this);
 return true;
}

void display() const {
 cout << name << ": " << A << "x + " << B << "y + " << C << "z + (" << D
<< ") = 0\n";
 cout << "VTPT: ";
 normalVector->display();
 cout << "\nDanh sach diem thuoc mat phang:\n";
 for (auto p : points) {
 cout << " + ";
 p->display();
 cout << "\n";
 }
}

};

int main() {
 Plane* P = new Plane("P", 2, 3, 4, -5);

 Point* p1 = new Point("M", 1, 1, 0);
 Point* p2 = new Point("N", -2, 3, 0);
 Point* p3 = new Point("K", 1, 2, 3);

 cout << "Them p1 vao (P): " << (P->addPoint(p1) ? "OK" : "FAILED") << "\n";
 cout << "Them p2 vao (P): " << (P->addPoint(p2) ? "OK" : "FAILED") << "\n";

 P->display();

 if (P->contains(p3)) {
 P->addPoint(p3);
 cout << "p3 thuoc (P), da lien ket thanh cong\n";
 } else {
 cout << "p3 khong thuoc (P), khoang cach: " << P->distanceTo(p3) << "\n";
 }

 delete p1;
 delete p2;
 delete p3;
 delete P;

 return 0;
}

```